

UI Automation Report

Name : Shahbaz Ahmad

GitHub branch: shahbaz_assessment_29Jun26

Date: 29-06-2026

Question 3

OBJECTIVE

To design and implement a hybrid automation framework combining API (RestAssured) and UI (Selenium + Cucumber) testing ensuring end-to-end validation. As a user, I want to apply filters and sorting so I can refine product search results.

Flow

1. Search → “face wash”
2. Apply filter:
 - * Price range
 - * Category/Brand
3. Apply sorting:
 - * Price low → high
4. Capture product list

Github Link:- [ust-sdet-training/SDET at shahbaz_assessment_29Jun26](#)

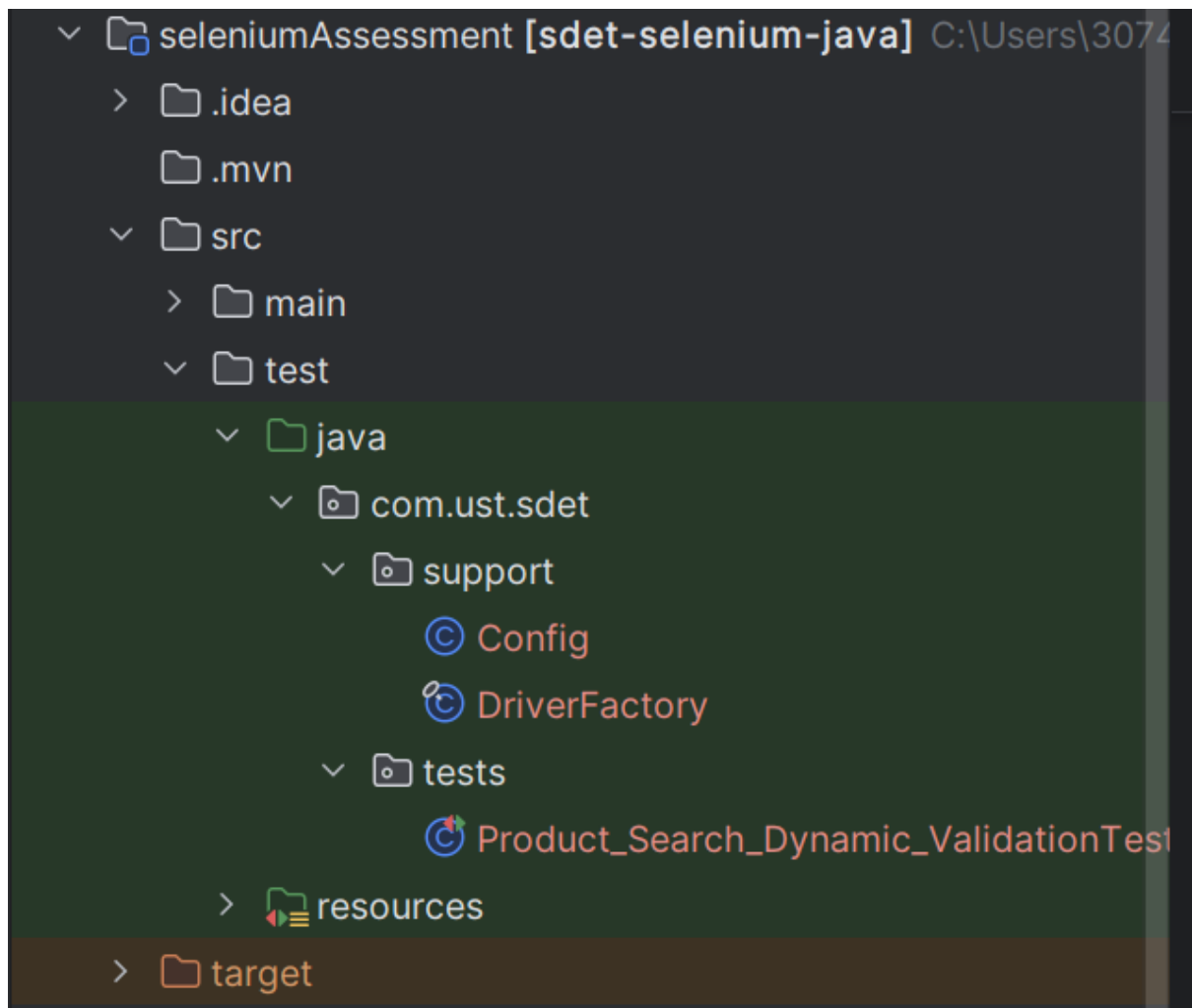
TOOLS & TECHNOLOGIES

Java, Maven, Selenium WebDriver, Cucumber BDD, RestAssured, IntelliJ / VsCode

FRAMEWORK WORKFLOW

1. Pass data to UI flow
2. Execute Selenium scenario
3. Validate response (UI)
4. Generate Allure Report

FOLDER STRUCTURE



SCENARIOS

As a user, I want to apply filters and sorting so I can refine product search results.

Flow

1. Search → “face wash”

2. Apply filter:

- * Price range

- * Category/Brand

3. Apply sorting:

* Price low → high

4. Capture product list

COMMANDS

Run full suite:

```
mvn clean test
```

Screenshot

DriverFactory

```
import java.util.HashMap;
import java.util.Map;

public final class DriverFactory { 4 usages
private DriverFactory(){} no usages

@ > public static WebDriver createDriver() { return createChromeD

@ public static WebDriver createChromeDriver(){ 2 usages
    ChromeOptions options = new ChromeOptions();
    Map<String, Object> prefs = new HashMap<>();

    prefs.put(k: "credentials_enable_service", v: false);
    prefs.put(k: "profile.password_manager_enabled", v: false);
    prefs.put(k: "profile.password_manager_leak_detection", v: false);
    options.setExperimentalOption(name: "prefs", prefs);
    if(Config.headless())
    {
        options.addArguments("--headless=new");
    }
    return new ChromeDriver(options);
}
}
```

Config.java

```
1 package com.ust.sdet.support;
2
3 public class Config { 2 usages
4
5     @ public static String baseUrl() 2 usages
6     {
7         return "https://www.purplle.com";
8     }
9     public static boolean headless() 1 usage
10    {
11        return false;
12    }
13 }
14
```

Product_Search_Dynamic_ValidationTest

```
package com.ust.sdet.tests;

import com.ust.sdet.support.DriverFactory;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.openqa.selenium.By;
import org.openqa.selenium.Keys;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

import java.time.Duration;
import static com.ust.sdet.support.Config.baseUrl;
```

```
public class Product_Search_Dynamic_ValidationTest {
```

```
    private WebDriver driver; 18 usages
```

```
    @BeforeEach
```

```
    void setup() { driver = DriverFactory.createChromeDriver(); }
```

```
    @AfterEach
```

```
    void tearDown() {
```

```
        if (driver != null) {
            driver.quit();
        }
```

Font size: 12pt

```

17 public class Product_Search_Dynamic_ValidationTest {
25     @AfterEach
26     void tearDown() {
27         if (driver != null) {
28             driver.quit();
29         }
30     }
31     @Test
32     @DisplayName("As a user, I want to search for beauty products and validate product details so that I can choose relevant products")
33     void purple() throws InterruptedException
34     {
35         WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
36
37         //1. Navigate → https://www.purple.com
38         driver.get(baseUrl());
39
40
41         //1. Search → "face wash"
42         var search :By = By.cssSelector("input[placeholder='What are you looking for?']");
43         driver.findElement(search).isDisplayed();
44         driver.findElement(search).click();
45
46         driver.findElement(search).sendKeys(...keysToSend: "face wash");
47         driver.findElement(search).click();
48         driver.findElement(search).sendKeys(Keys.ENTER);

```

```

17 public class Product_Search_Dynamic_ValidationTest {
33     void purple() throws InterruptedException
46         driver.findElement(search).sendKeys(...keysToSend: "face wash");
47         driver.findElement(search).click();
48         driver.findElement(search).sendKeys(Keys.ENTER);
49         driver.get(baseUrl()+" /search?q=face%20wash&is_form_search=1");
50
51
52         //2. Apply filter:
53         // * Price range
54         // * Category/Brand
55         //3. Apply sorting:
56         wait.until(ExpectedConditions.urlContains( fraction: "face"));
57         driver.navigate().to(driver.getCurrentUrl());
58         wait.until(ExpectedConditions.urlToBe(baseUrl()+" /search?q=face%20wash&is_form_search=1"));
59
60         var brand :By =By.cssSelector("#filterHeader > div > div > ul > li:nth-child(2) > a > span.tx-blk1i");
61         wait.until(ExpectedConditions.visibilityOfElementLocated(brand));
62         driver.findElement(brand).isDisplayed();
63         driver.findElement(brand).click();
64
65
66         // * Price low → high
67         var price :By = By.cssSelector("#filterHeader > div > div > ul > li:nth-child(3) > a > span.tx-blk1i");
68         wait.until(ExpectedConditions.visibilityOfElementLocated(price));
69         driver.findElement(price).isDisplayed();

```

```

17     public class Product_Search_Dynamic_ValidationTest {
33         void purple() throws InterruptedException
62             driver.findElement(brand).isDisplayed();
63             driver.findElement(brand).click();
64
65
66             // * Price low → high
67             var price :By = By.cssSelector("#filterHeader > div > div > ul > li:nth-child(3) > a > span.tx-blk1");
68             wait.until(ExpectedConditions.visibilityOfElementLocated(price));
69             driver.findElement(price).isDisplayed();
70             driver.findElement(price).click();
71
72
73             // //4. Capture product list
74             var list_firstProduct :By = By.cssSelector("#listing_item_353947 > div.m-0.mt-3.position-relative.n
75             wait.until(ExpectedConditions.visibilityOfElementLocated(list_firstProduct));
76             driver.findElement(list_firstProduct).isDisplayed();
77
78         }
79     }

```

Result

```

[INFO] Compiling 3 source files with javac [debug target 22] to target\test-classes
[INFO] Annotation processing is enabled because one or more processors were found
on the class path. A future release of javac may disable annotation processing
unless at least one processor is specified by name (-processor), or a search
path is specified (-processor-path, -processor-module-path), or annotation
processing is enabled explicitly (-proc:only, -proc:full).
Use -Xlint:-options to suppress this message.
Use -proc:none to disable annotation processing.
[INFO] --- surefire:3.5.6:test (default-test) @ sdet-selenium-java ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.ust.sdet.tests.Product_Search_Dynamic_ValidationTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 14.95 s -- in com.ust.sdet.tests.Product_Search_Dynamic_ValidationTest
[INFO] Results:
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 22.656 s
[INFO] Finished at: 2026-06-29T17:26:21+05:30
[INFO] -----
PS C:\Users\307424\Documents\QA Training\Assessment\FirstIntrimTest\SDET\seleniumAssessment>

```

Explanation

- 1 First create driver factory to communicate with web
 - 2 Config to get base url
 - 3 perform test base on use story
1. Search → “face wash”
 2. Apply filter:

- * **Price range**

- * **Category/Brand**

3. Apply sorting:

- * **Price low → high**

4. Capture product list

CONCLUSION

Utilizes **Selenium WebDriver** for automating browser-based UI testing.

Enables validation of **user journeys and frontend workflows**.

UI components are modular, **and test steps**.

Easily extendable for new UI features without breaking existing tests.

